ToddySoft Connect ADS Driver - Ignition 8.3

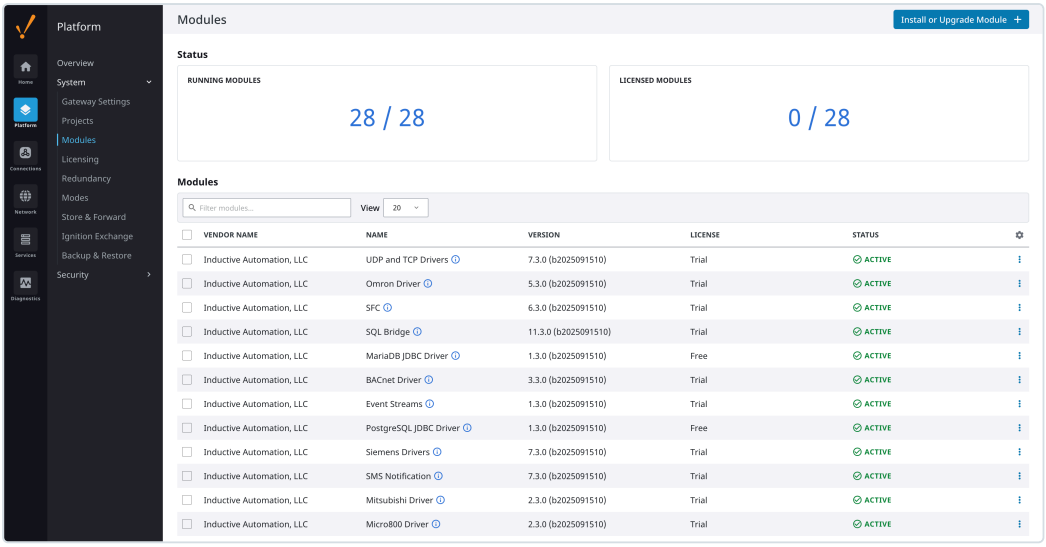
Ignition 8.3 module for connecting to Beckhoff TwinCAT PLCs via the ADS/AMS protocol, with support for plain TCP and encrypted "Secure ADS" (TLS) connections.

The device configuration uses a custom configuration form that adapts to the selected **Transport**: only the settings relevant to the chosen transport are shown, so you are never presented with fields that do not apply.

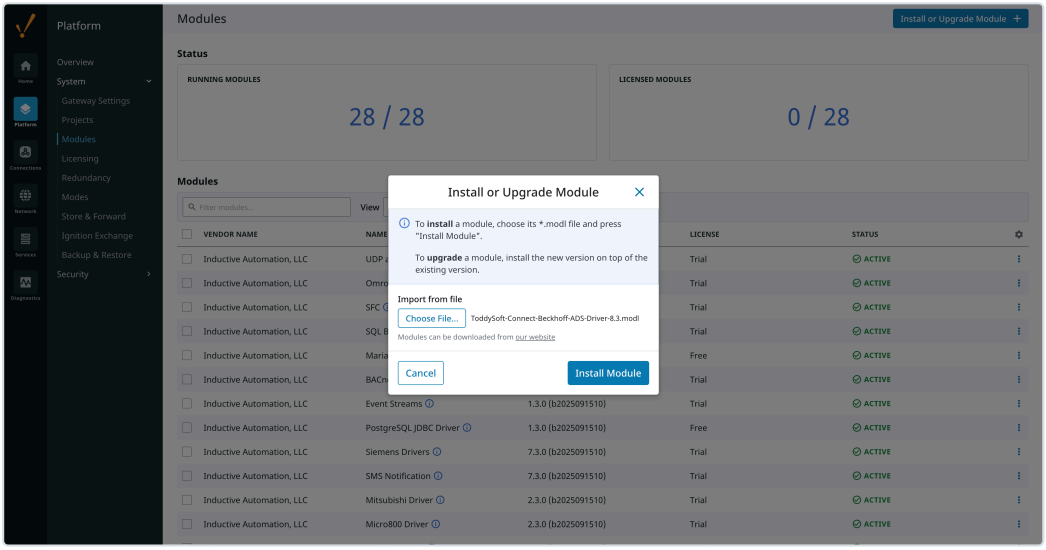
Installation

> **This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `…-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. This module declares the dependency as **required** — a mechanism 8.3 has and 8.1 lacks — so Ignition does not attempt to load it at all, and reports the reason instead of failing with a stack trace. In **Status > Diagnostics > Logs** you will see:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> This is the same handling Ignition's own drivers get in the same situation.

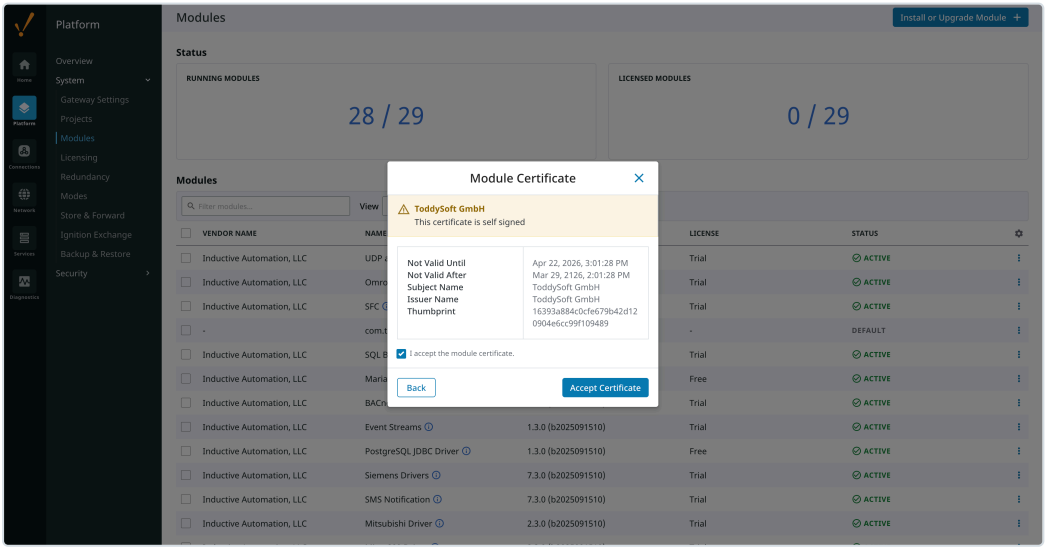
1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.



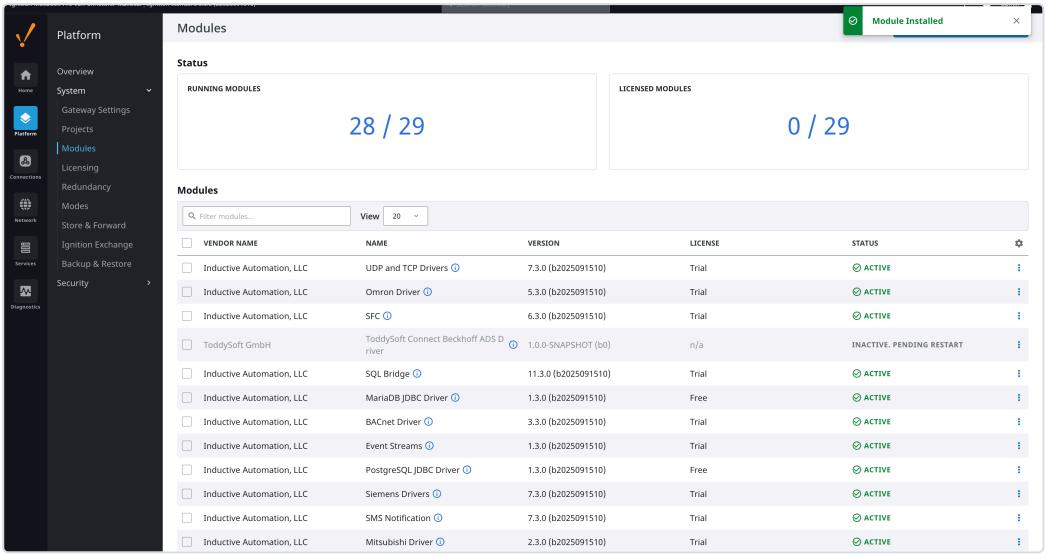
2. Click **Install or Upgrade a Module...**.



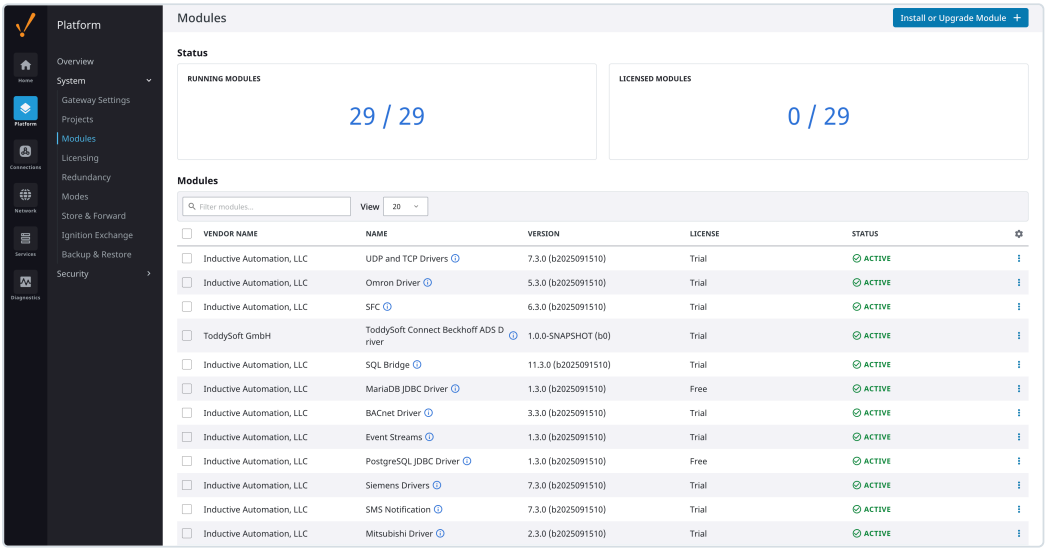
3. Select the `ToddySoft-Connect-Beckhoff-ADS-Driver-8.3.modl` file and upload it.



4. Accept the license / certificate prompt to continue the installation.

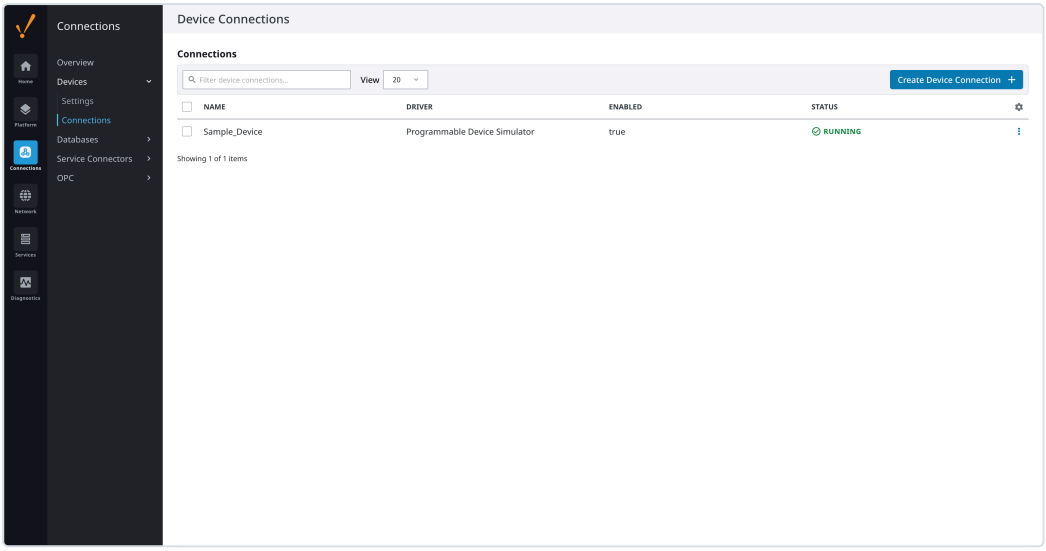


5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.

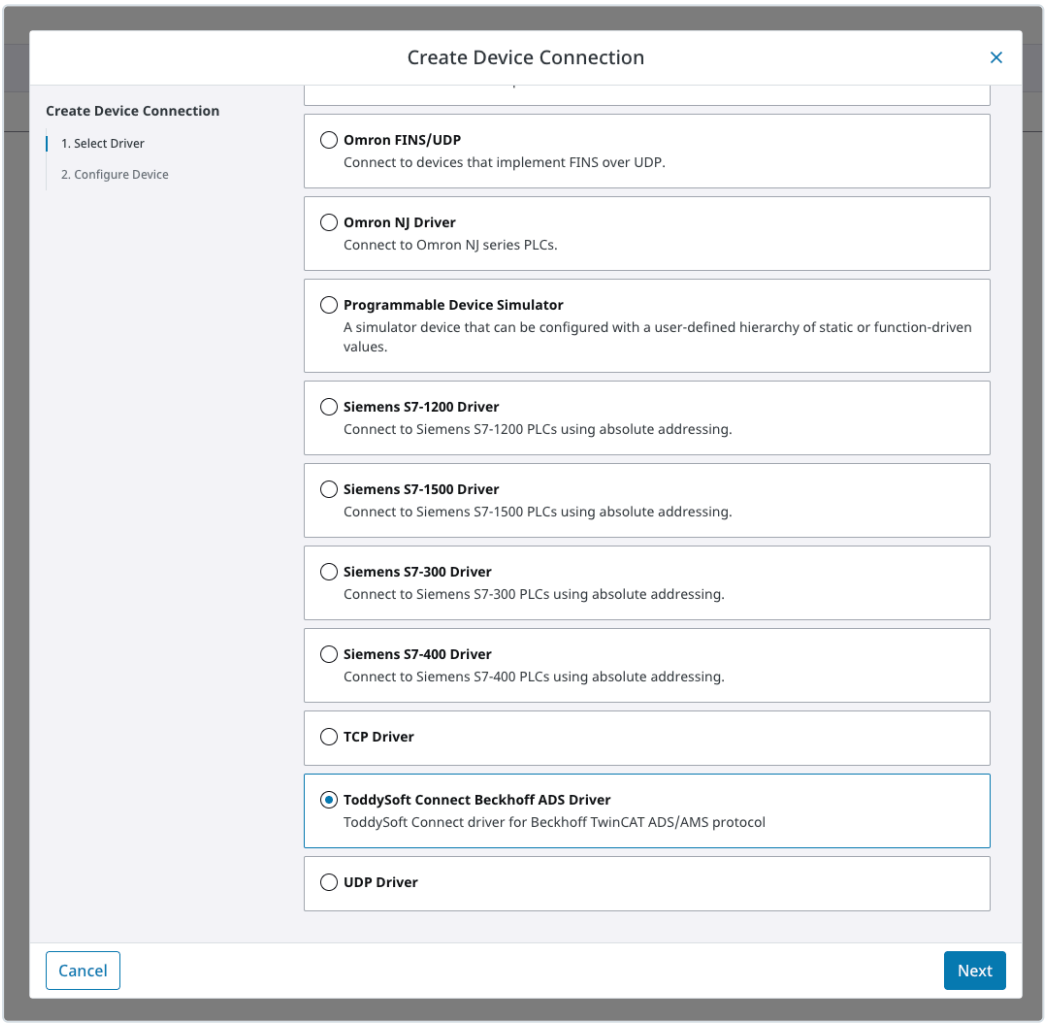


Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

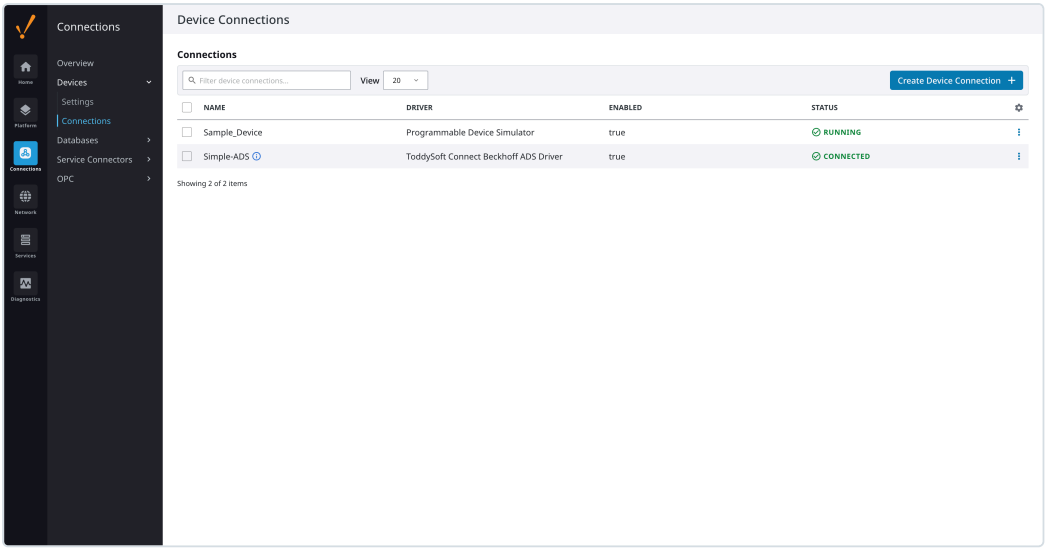


2. Click **Create new Device...** and select **ToddySoft Connect ADS Driver** as the device type.



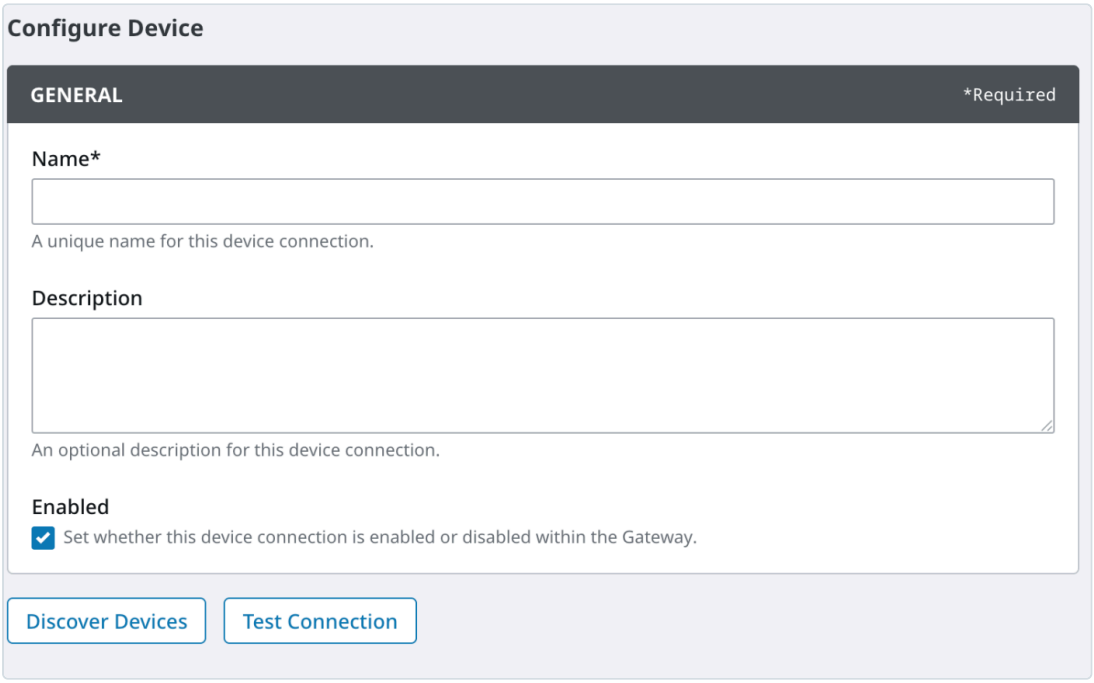
3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

4. The new device then appears in the Device Connections list.



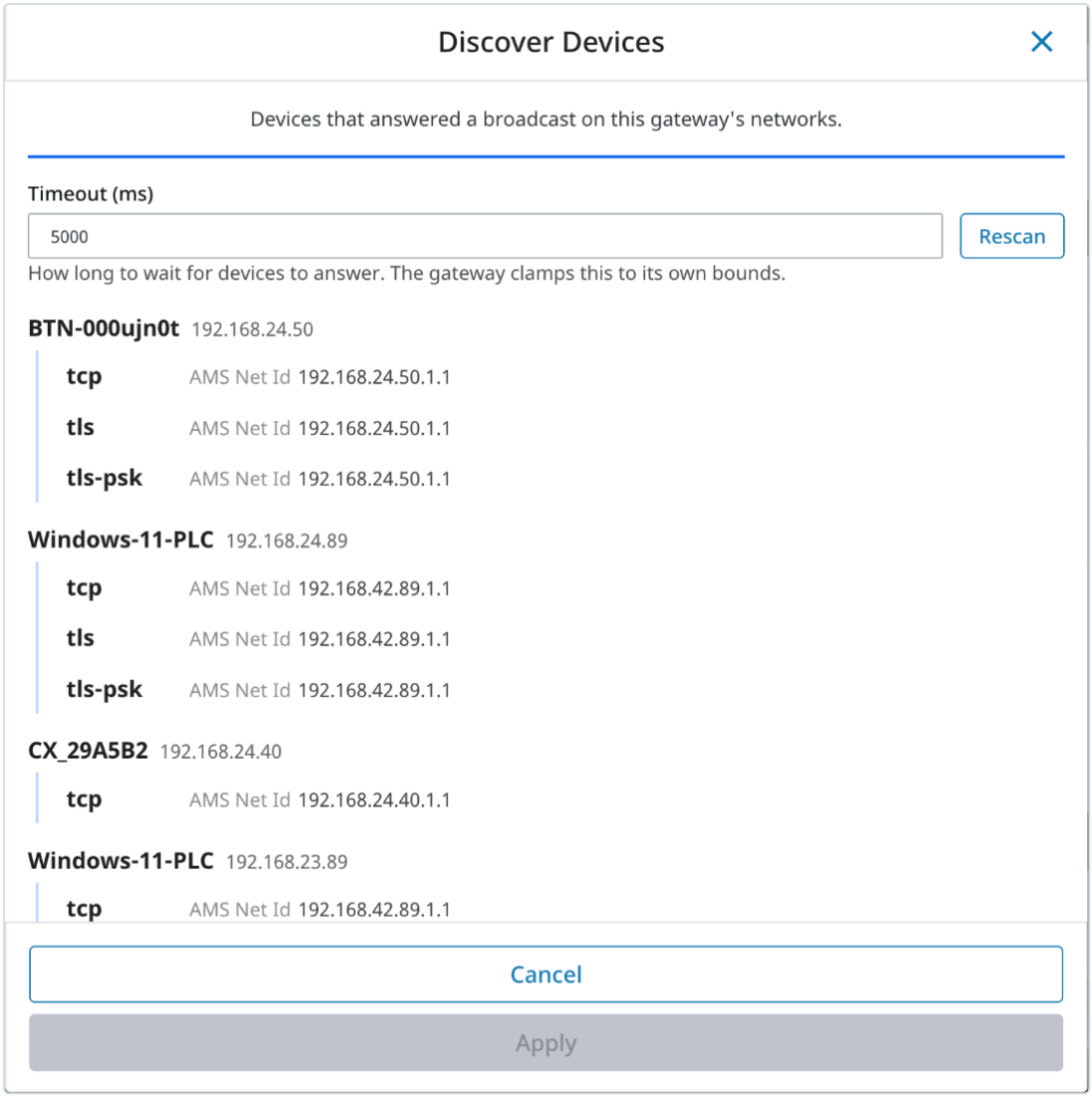
Finding devices, and checking one before you save it

Above the configuration form sits a row of actions: **Discover Devices**, **Test Connection** and, once a check has found something to fix, the setup actions that repair it. They are at the top on purpose — the form is long, and none of this should require scrolling to the end of it.



Discover Devices

Sends an ADS broadcast on the gateway's own networks and lists the controllers that answer. Results are grouped by device, with one row per transport that device offers, so a controller reachable over plain AMS, Secure ADS and TLS-PSK appears once with three ways in.



> **In Docker, discovery needs the discovery agent.** A container sits on Docker's network, not on the plant network the controllers are on, so the search alone finds nothing there. See Running in Docker: the discovery agent; without it, enter the host and AMS Net ID by hand — everything else on this page, **Test Connection** included, works normally.

Selecting a row fills in the form: host, port, target AMS Net ID and AMS port, and the transport itself. Three things it deliberately does not do:

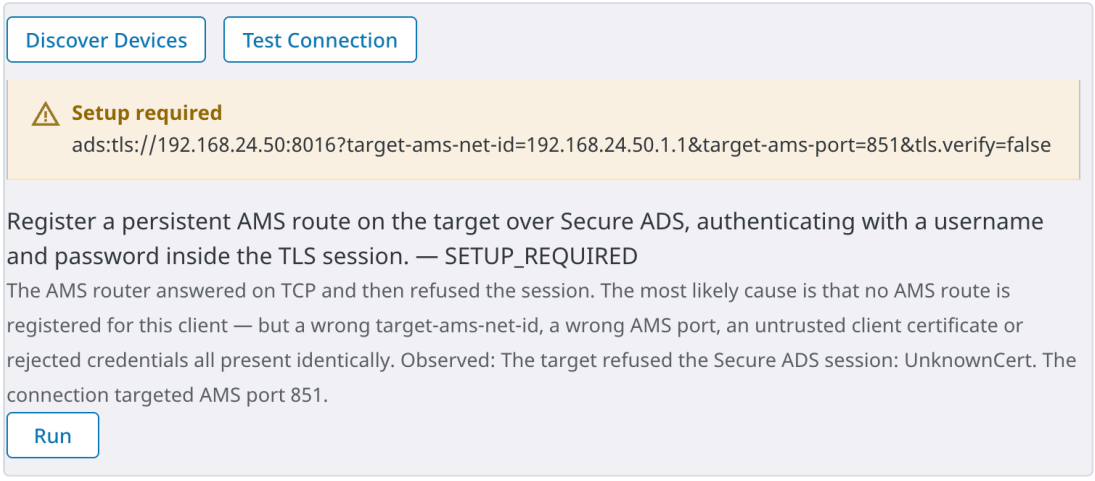
- **It fills in only what the device actually reported.** A field the broadcast said nothing about is left alone rather than guessed at.
- **It never fills a password, key or keystore.** A broadcast tells you where a device is, never how to authenticate to it.
- **It does not fill in Source AMS Net ID or Source AMS Port.** Those describe *this gateway*'s end of the connection, not the device, and the value a broadcast reports comes from whichever network interface happened to receive the reply — on a

gateway with several addresses, easily the wrong one. Left empty, the driver works them out from the connection it actually opens.

If applying a result would overwrite something you have already typed, it lists what would change and asks first. Running a sweep needs permission to create device connections; being logged in is not enough.

Test Connection

Asks the driver to diagnose the configuration exactly as entered — using the connection string the device itself would use, with secrets removed — and reports what it found rather than a bare success or failure. An unreachable device is reported as unknown rather than as broken, because those are different problems.



Where the driver can repair what it found, the diagnosis carries the fix. The common case is a Secure ADS device whose AMS route has not been registered on the controller: the panel offers to register it, asking for the controller's administrator account. **Those credentials are used for that one call and are never stored** — not in the device configuration, not in the gateway's database.

Some fixes cannot be automated. A TLS-PSK route is one: TwinCAT will not accept it over the wire, and attempting it damages the route. In that case the panel prints the steps to do it by hand instead of offering a button that cannot work.

Registering an AMS route

A Beckhoff controller only answers a host it has an **AMS route** for. Until this release that route had to be created on the controller by hand — in TwinCAT XAE, or by editing `/etc/TwinCAT/3.1/Target/StaticRoutes.xml` — before an Ignition device could connect at all. The module can now do it for you.

When **Test Connection** finds that the route is missing, it offers **Register AMS route** on that finding. The dialog asks for:

Field	Notes
Route Name	The name the route is filed under on the controller. Pre-filled from this gateway's certificate, which is what the controller checks — usually leave it as it is.
Route Username	An administrator account **on the controller**, not an Ignition user. On a stock TwinCAT system this is `Administrator`.
Route Password	That account's password.

ads-route-tls

Register a persistent AMS route on the target over Secure ADS, authenticating with a username and password inside the TLS session.

Route Username

Account on the target permitted to add routes, typically 'Administrator'.

Route Password

Password for that account. Transmitted inside the TLS session.

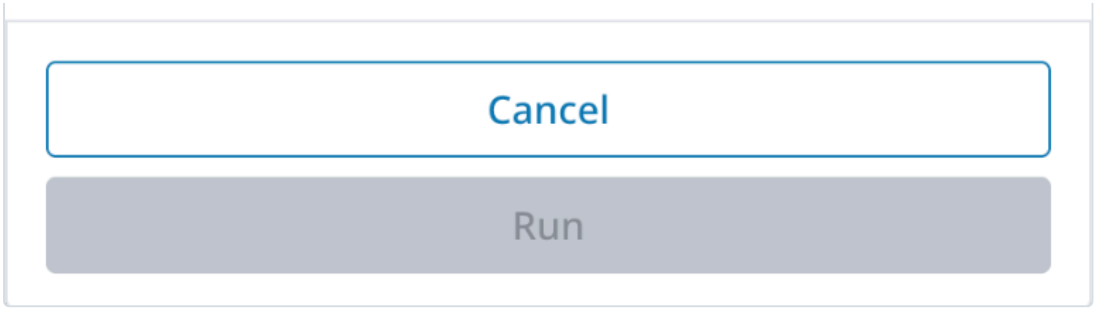
Ignore Common Name

Registers the route with the certificate common-name check waived, by setting the IGNORE_CN flag (bit 6) on the registration message — flags 0x00F0 instead of 0x00B0. The target records it as a property of the route: the entry in StaticRoutes.xml becomes <Tls SelfSigned="true" IgnoreCn="true">. Off by default, because it relaxes a check that exists to bind a route to one certificate identity; turn it on only when the announced machine name genuinely cannot be made to match the certificate's CN. Prefer setting the driver's machine-name parameter instead.

Route Name

MacBook-Pro-von-Christofer-M2

Name the route is registered under on the target. Defaults to this machine's hostname, which is what an engineering tool would use.
Must match this gateway's certificate, whose common name is `MacBook-Pro-von-Christofer-M2`. The target checks the certificate, not the name you give the route.



Those credentials are used for that single call and are **never stored** — not in the device configuration, not in the gateway's database, and they never appear in the connection string or the logs. If the route is later removed from the controller, you will be asked for them again.

Once the route is registered, run **Test Connection** again: the finding should clear and the device can be created.

Two limits worth knowing:

- **TLS-PSK routes cannot be registered this way.** TwinCAT will not accept a pre-shared key over the wire, and attempting it *empties the key on an existing route* — so the module refuses to try and shows the manual steps instead. Create a PSK route in TwinCAT.
- **A route is bound to a certificate.** If this gateway's certificate is replaced — including the automatic regeneration described under *Version history* — a route registered against the old one no longer matches and must be registered again.

A note on names, for Secure ADS with a self-signed certificate

Three names have to agree, and the certificate is the one that decides:

- the **common name** of this gateway's certificate,
- the **Machine Name** announced to the controller (blank means this gateway's host name, without its domain),
- the **name the route is filed under** on the controller.

If they disagree, the controller refuses the session with `CnMismatch` — after a perfectly good TLS handshake, on every connection. The module fills the route name in from the certificate for you when registering a route, and the **Machine Name** field is there for the case where the controller's route is already filed under something else.

Running in Docker: the discovery agent

A gateway in a Docker container sees only the container's networks, not the plant's: *Discover Devices* sends a UDP broadcast, which never leaves the container's network, so no controller answers. The **ToddySoft discovery agent** closes that gap. It is a small, separate container that runs on the same machine with the host's network and searches the plant networks on the gateway's behalf. The agent only searches:

connecting, browsing and reading stay with the gateway, through the machine's normal routing. A gateway installed directly on the plant machine, or a container on the host network, does not need it.

Installing it. The agent comes as an image archive next to this module, in the share you download the module from: `discovery-agent-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the agent of the module's version. On the machine that runs the Ignition container (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lets it search the plant networks, and `NET_RAW` lets it send and capture the discovery frames. Docker Desktop (macOS, Windows) runs containers in a virtual machine, whose networks the agent would search instead of the plant's.

Nothing to set in Ignition. On every **Discover Devices** the module looks for the agent at its container's default gateway, which on Docker is the machine itself. An agent started later is found on the next search, with no restart. A line above the results says what happened:

- *via discovery agent*: the agent searched; its devices are listed together with any the gateway found itself, each once.
- *no discovery agent found · container networks only*: no agent runs; the search is the gateway's own, as before.
- *discovery agent refused: …* or *discovery agent not usable: …*: the reason follows, for example the licence or the shared secret below.

Connecting still needs the Source AMS Net ID. The agent finds the controller; the connection is the gateway's, from inside the container, so the Source AMS Net ID described under Docker related issues applies as before, and the controller needs its route.

Licence. The agent searches only for a gateway licensed for this driver. With each search the module sends the licence it runs on, the bundled trial included, over an encrypted connection; the agent checks it, uses it for that search and keeps nothing.

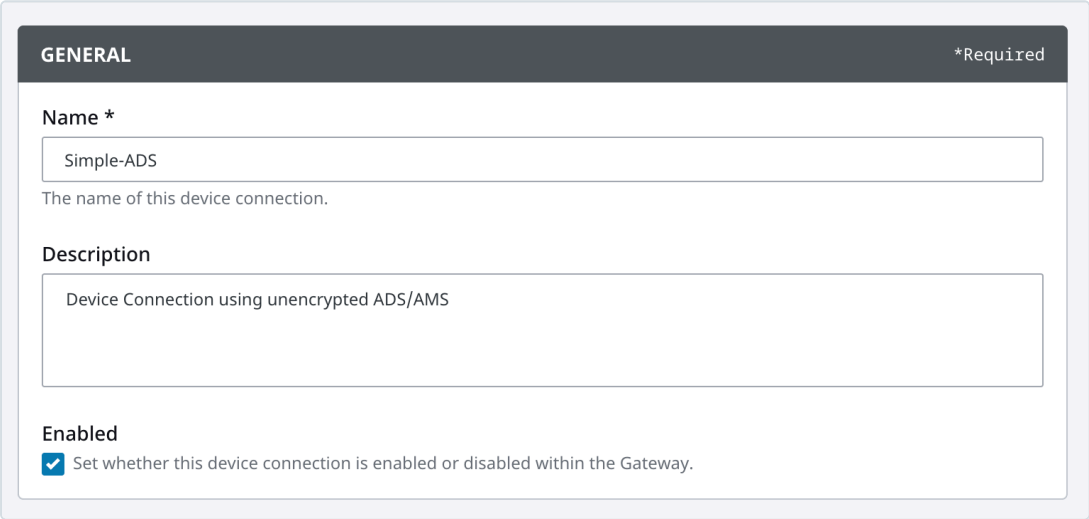
Security. The agent listens only on the Docker networks' own addresses on that machine (port 47900), never on a plant network, and accepts connections only from the container networks. To restrict it further, give it a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the Ignition container. With a secret set, a gateway without it, or with another, is refused. To stop the module looking for an agent, set `TODDYSOFT_DISCOVERY_AGENT=off` on the Ignition container.

Configuration

The configuration form is organized into the following sections, top to bottom: **General**, **Transport**, **Transport Settings**, **ADS / AMS Settings**, **Advanced Transport Settings** and **Audit Log**.

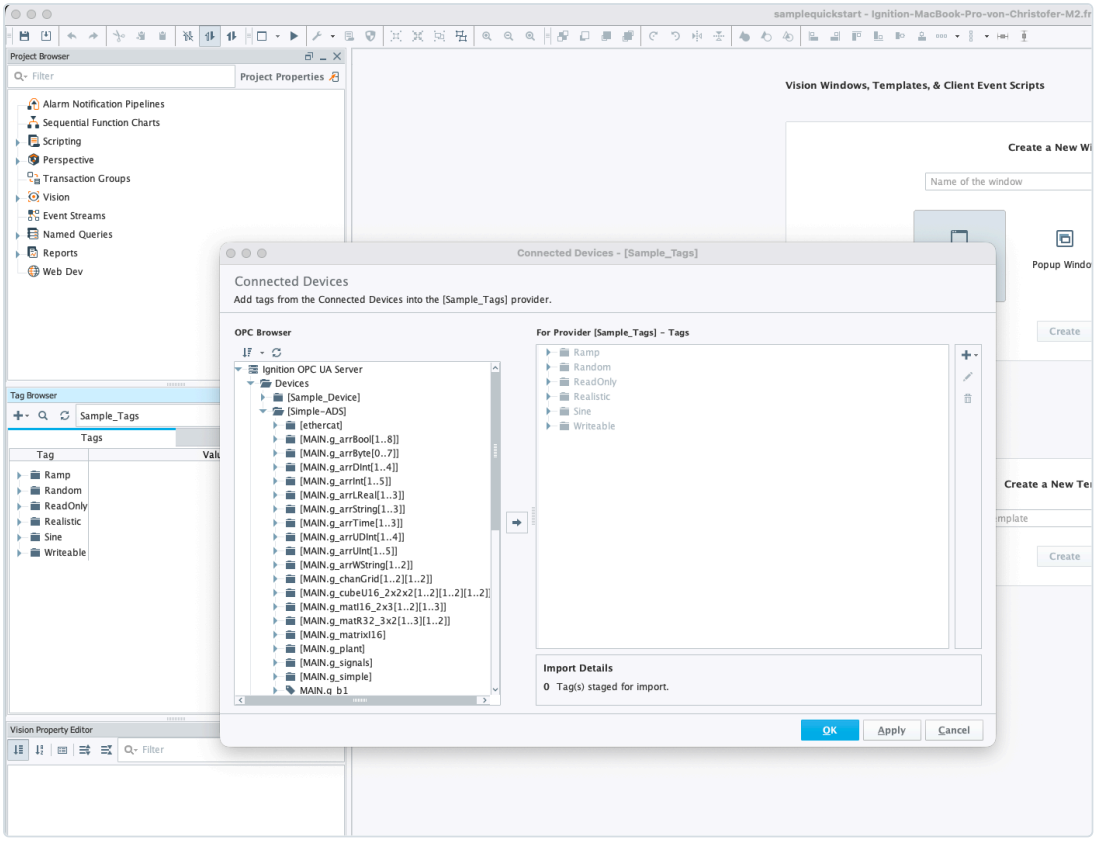
General

Assigns a name (and optional description) to the connection and controls whether it is enabled.



Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	`true`

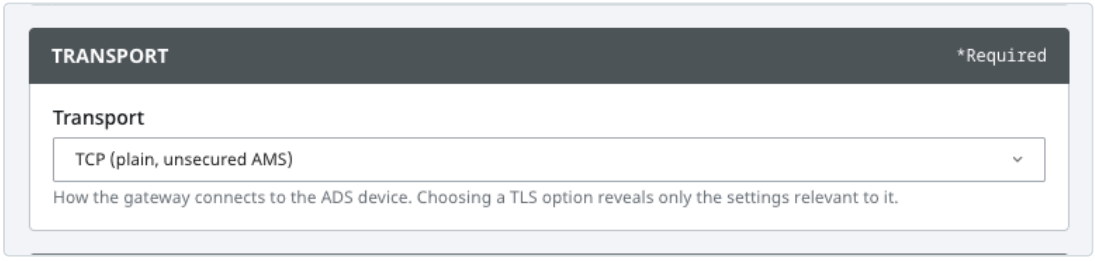
This is the name the connection is then referenced under in other parts of Ignition:



If `Enabled` is not checked, no connection is established until it is enabled.

Transport

Selects how the gateway connects to the ADS device. Choosing one of the TLS options reveals only the settings relevant to it in the **Transport Settings** section below.



Transport	Description
TCP (plain, unsecured AMS)	Plain, unencrypted ADS over TCP (default). No certificates or keys needed.
TLS-PSK	Encrypted ("Secure ADS") using a pre-shared key + identity. No certificates.
TLS-SSC (Self-Signed Certificate)	Encrypted using an automatically generated self-signed certificate.
TLS-SCA (Shared Certificate Authority)	Encrypted using certificates signed by a shared Certificate Authority (CA).

When any TLS transport is selected, the connection port automatically switches from `48898` to `8016`, unless a non-default port has been set explicitly.

> **The ADS route must already exist on the PLC.** Earlier versions registered a route as part of connecting, which is why this page used to ask for a **Route Name**, **Route Username** and **Route Password**. Those three settings are gone. Registering a route is a one-time administrative act that needs the controller's administrator credentials, and requiring them here forced those credentials into the configuration that every reading device carries for the life of the connection.
>
> Create the route on the controller before adding the device — in TwinCAT XAE, or by editing `/etc/TwinCAT/3.1/Target/StaticRoutes.xml` directly — and the device then connects with no administrator credentials at all. A device pointed at a controller that has no matching route will fail to connect.

> **Storing passwords and keys:** The secret fields used by the TLS transports (`Keystore Password` and `Pre-Shared Key`) are handled through Ignition 8.3's secret management. For each you can choose **None**, store the value **Embedded** (encrypted within the device configuration) or **Referenced** from a configured secret provider. Values are never persisted in plain text.

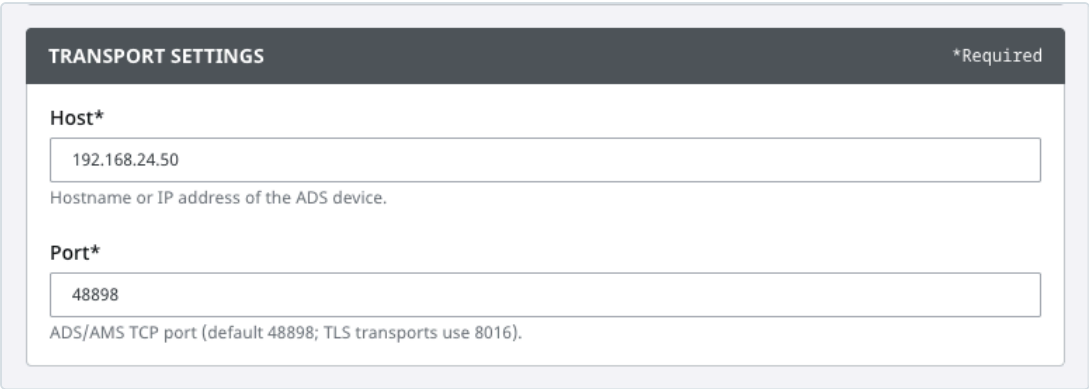
Transport Settings

Always contains **Host** and **Port**; the remaining fields depend on the selected transport.

Setting	Description	Default
Host	Hostname or IP address of the TwinCAT device	`localhost`
Port	TCP port number (default 48898; TLS: 8016)	`48898`

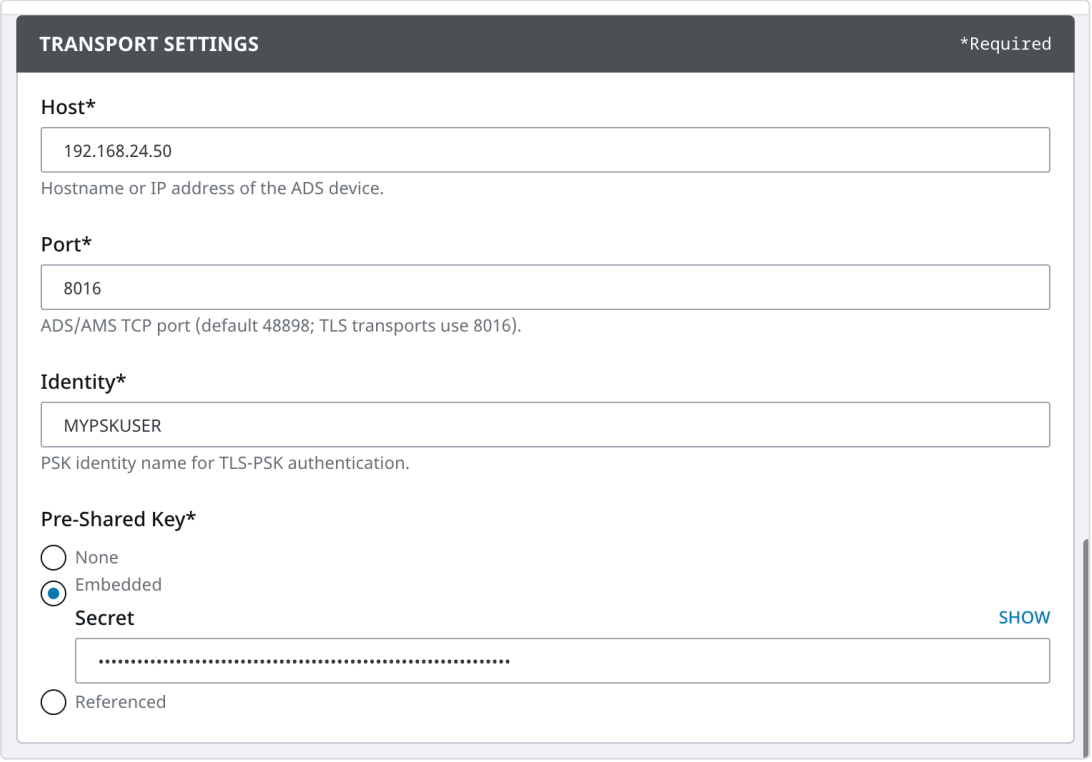
TCP (plain, unsecured AMS)

Plain TCP shows only Host and Port — no security fields.



TLS-PSK

Uses TLS-PSK, where both sides share a secret key and identity string. No certificates are needed — this is the simplest way to encrypt ADS communication.



Setting	Description	Default
Identity	PSK identity string (must match the PLC configuration)	*(empty)*
Pre-Shared Key	Hex-encoded shared secret key (secret field)	*(None)*

Configuring the PLC for PSK: the PLC must be configured with matching PSK settings. In the TwinCAT Secure ADS configuration (usually in `/etc/TwinCAT/3.1/Target/StaticRoutes.xml`):

- Set the **Identity** to match the value in Ignition (e.g., `MYPSKUSER`)
- Set the **Password/Key** that will be used to derive the pre-shared key

```
                <Server>
                        <Tls>
                                <Psk>
                                        <Identity>MYPSKUSER</Identity>
                                        <Pwd>MySecret</Pwd>
                                </Psk>
                        </Tls>
                </Server>
        </RemoteConnections>
</TcConfig>
```

TwinCAT derives the PSK key by computing `SHA-256(Identity + Password)`. Note that the Identity must be **all uppercase**. To calculate the hex key needed for the Ignition configuration:

```
echo -n "<Identity><Password>" | shasum -a 256
```

For example, with Identity `MYPSKUSER` and Password `MySecret`:

```
echo -n "MYPSKUSERMySecret" | shasum -a 256
# Result:
        a50d3154fea7bb54eec6710164e33ced872223f0a0ff05179e93e89e7f76c6f0
```

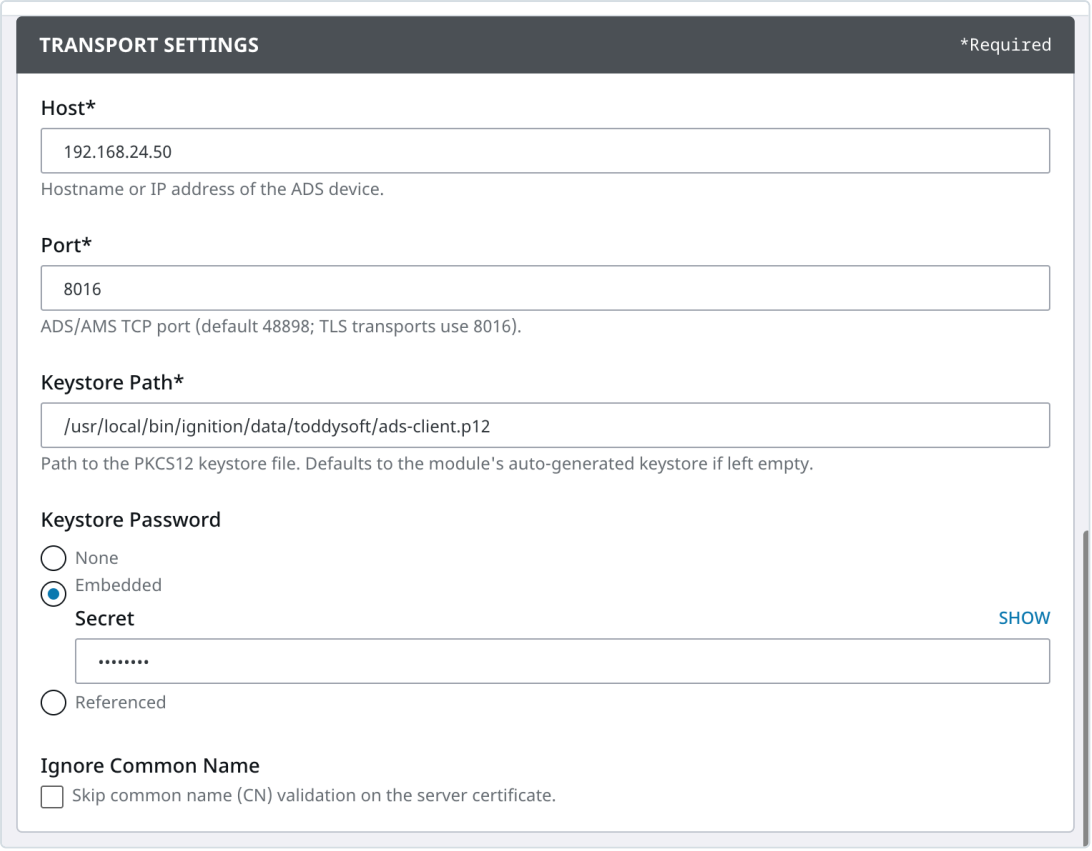
Use the resulting 64-character hex string as the **Pre-Shared Key** value in Ignition (stored Embedded or Referenced).

The corresponding route entry in `/etc/TwinCAT/3.1/Target/StaticRoutes.xml` looks like this:

```
<Route>
        <Name>MyDevMachine</Name>
        <Address>192.168.24.220</Address>
        <NetId>192.168.24.220.1.1</NetId>
        <Type>TCP_IP</Type>
        <Tls>
                <Psk>
                        <Identity>MYPSKUSER</Identity>
                        <Key>a50d3154fea7bb54eec6710164e33ced872223f0a0ff05179e93e89e7f76c6f0</Key>
                </Psk>
        </Tls>
</Route>
```

TLS-SSC (Self-Signed Certificate) and TLS-SCA (Shared Certificate Authority)

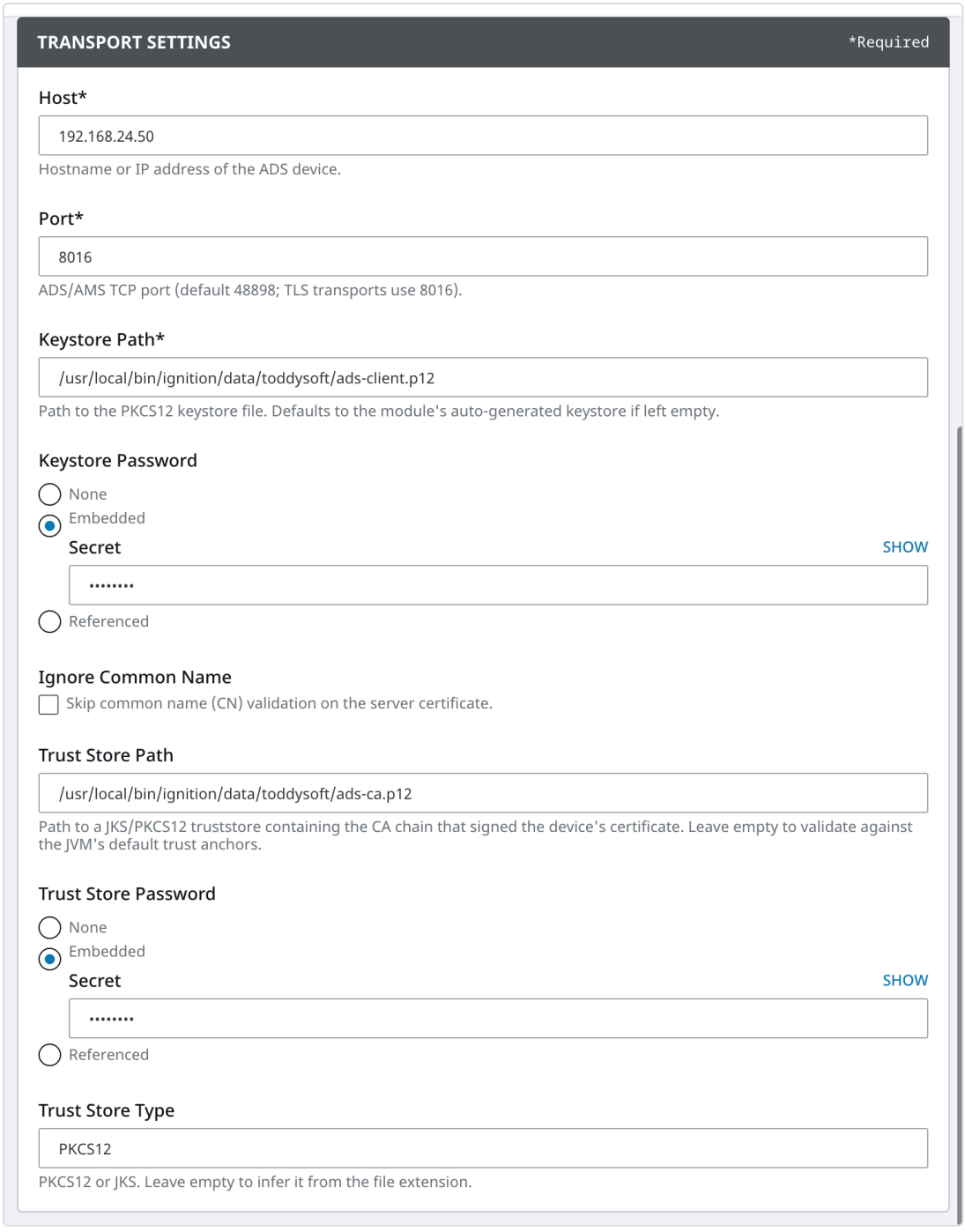
The two certificate-based transports share most of their fields. **TLS-SSC** uses an automatically generated self-signed certificate (the module manages the PKCS12 keystore for you; the ADS route itself must already exist on the controller). **TLS-SCA** uses certificates signed by a shared Certificate Authority that both Ignition and the PLC trust — the most secure option for production.



Setting	Description	Default
Keystore Path	Path to the PKCS12 keystore file. For TLS-SSC, leave empty to use the module's auto-generated keystore; for TLS-SCA, point to the CA-signed certificate.	*(auto / required)*
Keystore Password	Password for the keystore (secret field)	*(None)*
Machine Name	Name announced to the controller in the Secure ADS session. Leave empty to announce this gateway's host name without its domain. With a self-signed certificate it must equal the certificate's common name.	*(host name)*
Trust Store Path	*TLS-SCA only.* Path to a JKS/PKCS12 truststore holding the CA chain that signed the PLC's certificate	*(JVM defaults)*
Trust Store Password	*TLS-SCA only.* Password for the truststore (secret field)	*(None)*
Trust Store Type	*TLS-SCA only.* `PKCS12` or `JKS`	*(from extension)*

The three **Trust Store** fields appear only when **TLS-SCA** is selected, because only that transport verifies the certificate the PLC presents. Note the direction: the **Keystore** holds *this gateway's own* certificate, which it presents to the PLC, while the **Trust Store** holds the CA certificate used to check the *PLC's* certificate.

Selecting **TLS-SCA** therefore shows three more fields than the form above:

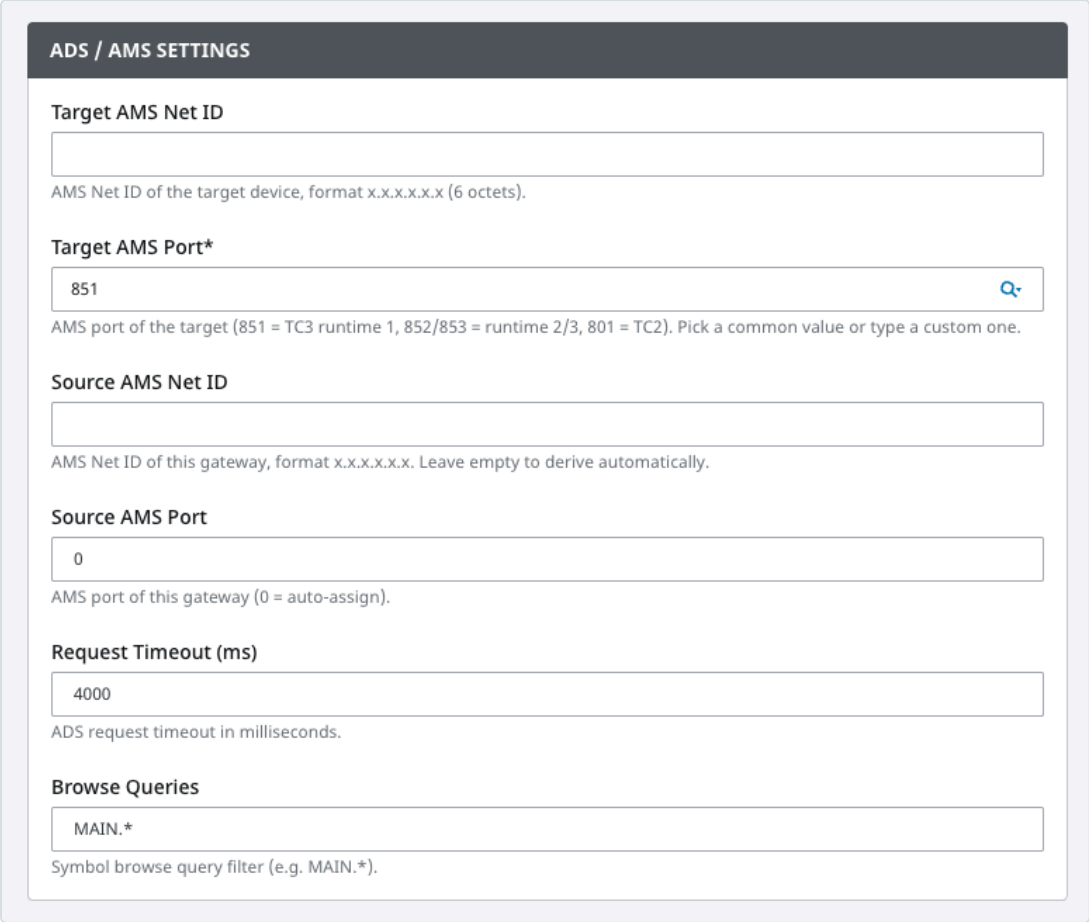


If your CA is a private one (that is, not among the public authorities the Java runtime already trusts), set **Trust Store Path** to a truststore containing it. Left empty, the driver validates against the Java runtime's default trust anchors, which will reject a privately signed certificate.

When a keystore path is not specified (TLS-SSC), the module automatically generates a keystore in the Ignition module settings directory. The matching route in `/etc/TwinCAT/3.1/Target/StaticRoutes.xml` must already exist on the PLC.

ADS / AMS Settings

Configures the ADS-specific addressing. These settings apply regardless of the selected transport.



Setting	Description	Default
Target AMS Net ID	AMS Net ID of the target device (format `x.x.x.x.x.x`). Defaults to `{target-ip}.1.1`	*(auto)*
Target AMS Port	AMS port of the target PLC runtime — pick a common TwinCAT port or type a custom value	`851`
Source AMS Net ID	AMS Net ID of this gateway (format `x.x.x.x.x.x`). Leave empty to derive automatically	*(auto)*
Source AMS Port	AMS port of this gateway (`0` = auto-assign)	`0`
Request Timeout	Default timeout for ADS requests in milliseconds	`4000`
Browse Queries	Symbol browse query filter. Separate multiple queries with `;` (e.g. `MAIN.*;GVL.*`)	`MAIN.*`

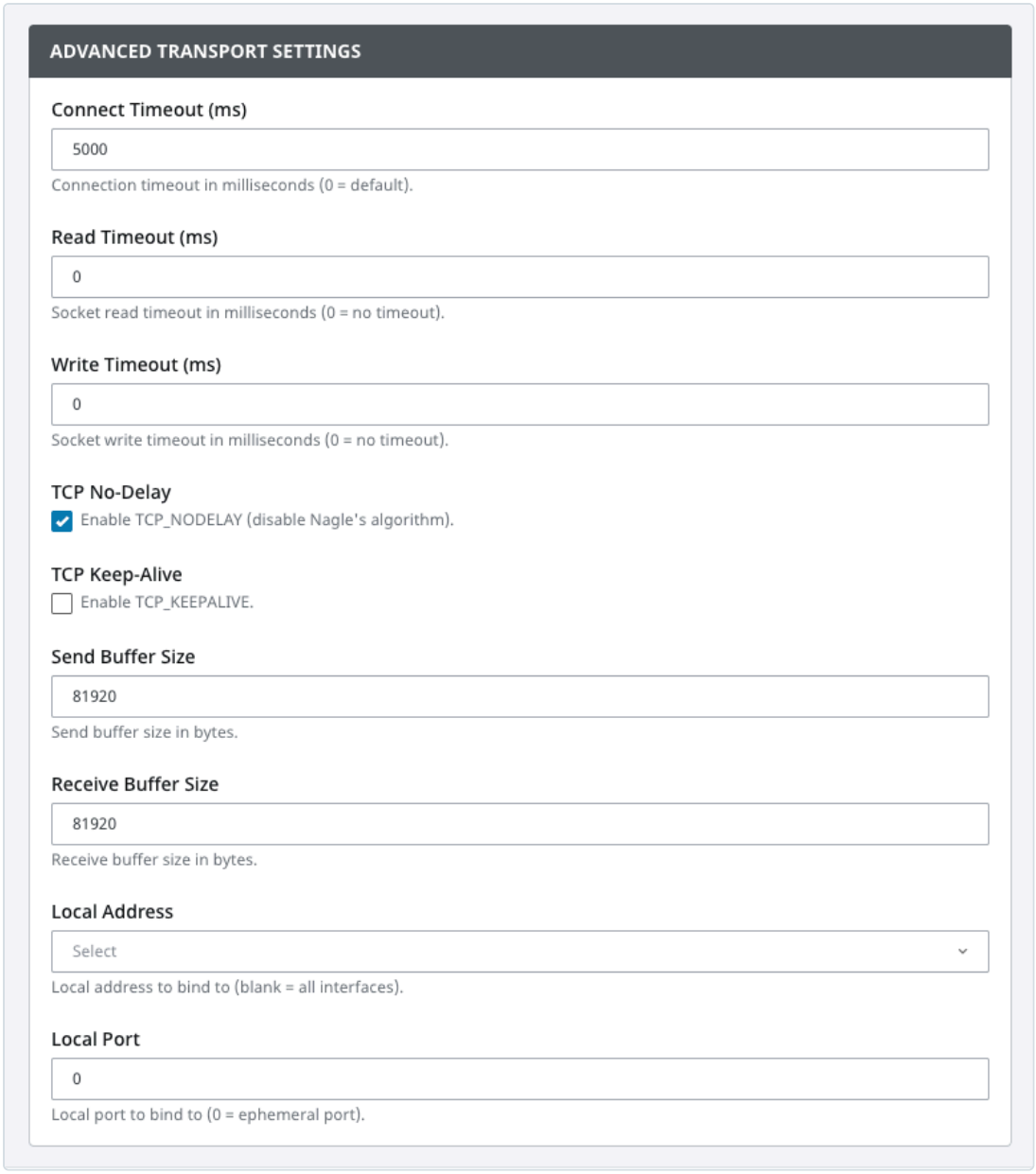
If not provided in the **Target AMS Net ID** field, the value is calculated by appending `.1.1` to the local IP address of the TCP connection. Note that there may be multiple paths to reach the PLC (if the device has multiple IP addresses). To control which local IP address is used, set the **Local Address** in the Advanced Transport Settings section.

The **Target AMS Port** field offers the common TwinCAT runtime ports as suggestions, each with a description, while still allowing any custom value:

Port	Description
851	TwinCAT 3 — PLC runtime 1 (default)
852 / 853	TwinCAT 3 — PLC runtime 2 / 3
801	TwinCAT 2 — PLC runtime 1
811 / 821 / 831	TwinCAT 2 — PLC runtime 2 / 3 / 4

Advanced Transport Settings

Lower-level TCP tuning options. These apply under whichever transport is active.



Setting	Description	Default
Connect Timeout	Connection timeout in milliseconds (0 = default)	`5000`
Read Timeout	Socket read timeout in milliseconds (0 = no timeout)	`0`
Write Timeout	Socket write timeout in milliseconds (0 = no timeout)	`0`
TCP No-Delay	Enable TCP_NODELAY (disable Nagle's algorithm)	`true`
TCP Keep-Alive	Enable TCP_KEEPALIVE	`false`
Send Buffer Size	Send buffer size in bytes	`81920`

Setting	Description	Default
Receive Buffer Size	Receive buffer size in bytes	`81920`
Local Address	Local address to bind to, chosen from the gateway's network interfaces (blank = all interfaces)	*(all)*
Local Port	Local port to bind to (0 = ephemeral port)	`0`

The **Local Address** field is a dropdown populated with the gateway machine's local IPv4 addresses, which is especially useful when the host has multiple interfaces (see the Docker and VPN notes below).

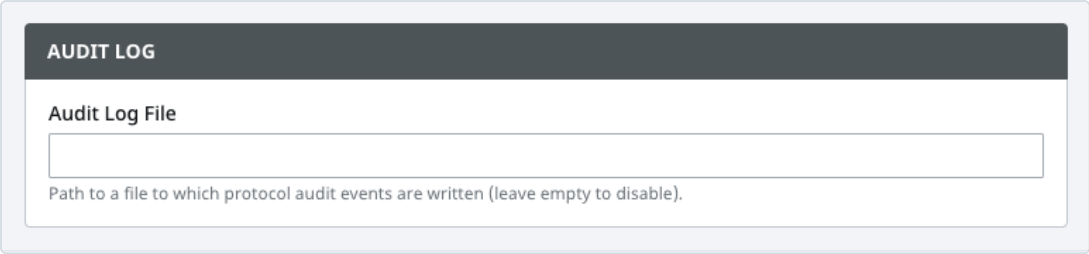
Limits

Ceilings on what the controller can make the driver allocate. Each one bounds something the *other* side chooses - a declared table depth, a nesting depth, the size of a sum command - so a device that misbehaves cannot exhaust the gateway. The defaults are the driver's own; raise one only for a controller measured to need it. Leaving them alone produces exactly the connection the device made before these settings existed.

Setting	Description	Default
Max Browse Depth	How deep the browser dereferences a symbol into child nodes. A type declared in terms of another that refers back to it forms a cycle that parses fine and then walks forever.	`20`
Max Data-Type Table Depth	Nesting depth accepted when parsing the data-type table uploaded from the device. Bounds how deeply the table is *parsed*, where Max Browse Depth bounds how deeply it is *dereferenced*.	`20`
Max Sum Sub-Commands	Sub-commands packed into one Sum-Read or Sum-Write; a larger request is split across several. The default is Beckhoff's own recommendation - the PLC collects and copies every variable in the request before it can start its next cycle.	`500`
Max Sum Bytes	Bytes one Sum-Read or Sum-Write may move through the AMS router; a larger request is split. The default is the AMS router's own limit.	`2097152`

Audit Log

The `Audit Log` writes important diagnostic information to a dedicated log file. This file can grow quickly, so it is advised to only enable it when investigating issues.



Setting	Description	Default
Audit Log File	Absolute path to the audit log file where events will be recorded. Leave empty to disable.	*(empty)*

NOTE: Please ensure the user the `Ignition` process is running under has write access to this path.

The Audit Log system automatically creates the file and rotates it (creating compressed files next to it) once a certain size is reached or on a time-based schedule.

Tag Addressing

Tags are addressed using symbolic names from the PLC program:

```
MAIN.bStart                    # Boolean
MAIN.iCounter                  # Integer
MAIN.fTemperature              # Float
GVL.sStatus                    # String
MAIN.stMotor.fSpeed            # Struct member
MAIN.aValues[0]                # Array element
MAIN.stArray[1].nField         # Struct array element field
```

Configure **Browse Queries** (e.g., `MAIN.*`) to control which symbols appear when browsing tags in the Ignition Designer.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open the device's settings (**Config > OPC UA > Device Connections**, edit the device, or create a new one): the **Licence** card at the top of the form shows the diagnosis. It is on the form for a new device too, so the licence can be checked before the first device exists.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

Cause	Meaning	What to do
`IGNITION_UNLICENSED`	Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence.	Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation. If a key was activated and this cause remains, see Troubleshooting: an older, expired key covering this module may be the one Ignition judges it by.
`NO_DATA_FOR_DRIVER`	The activated key carries no ToddySoft licence for this driver.	Send the report to ToddySoft.
`DATA_INCOMPLETE`	The licence data arrived incomplete or damaged.	Send the report to ToddySoft; the licence has to be delivered again.
`DATA_REJECTED`	The driver refused the delivered licence; the reason is shown.	Send the report to ToddySoft.

Cause	Meaning	What to do
`RUNNING_ON_BUNDLED_TRIAL`	A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends.	Send the report to ToddySoft.
`NOT_CHECKED`	Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded.	Send the report to ToddySoft.
`VALID`	Licensed; tier and expiry are shown.	Nothing.
`NOT_YET_KNOWN`	The first seconds after start-up, before Ignition reports a licence state.	Wait a moment.

Sending a report to support. The diagnosis comes as a plain-text report, the one thing to send with any licence question. **Copy report** puts it on the clipboard; **Show report** displays it. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.ads/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

Troubleshooting

Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index", or `IGNITION_UNLICENSED` with "trial expired", in the licence diagnosis:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one. Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.

- If one of the two keys has **expired**, Ignition may judge the module by that key alone: the diagnosis shows `IGNITION_UNLICENSED` ("trial expired - no activated licence applies to this module") and "Data: none received", although the new key was activated successfully.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Platform > Licensing**, then restart the module (or the gateway). In a redundant pair, check every node.

The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

Device shows "Faulted":

- Verify the PLC hostname/IP is reachable
- Check that AMS Net IDs match the TwinCAT configuration
- Ensure port 48898 (or 8016 for TLS) is not blocked by a firewall
- Review Gateway logs: **Status > Diagnostics > Logs**

Tags show bad quality:

- Verify the variable exists in the PLC program
- Check spelling and case sensitivity
- Ensure the PLC program is running

TLS connection fails:

- For TLS-PSK: verify the identity and key match on both sides
- For TLS-SSC: check that route credentials are correct
- For TLS-SCA: ensure both certificates are signed by the same CA

Docker related issues

When running `Ignition` inside `Docker`, the host receives an IP address on the Docker-internal network. When connecting to the PLC, the connection originates from one of the host's IP addresses. In this case the connection needs a **Source AMS Net ID** that refers to the host's IP address (and the **Local Address** in Advanced Transport Settings can be used to pin the originating interface).

Leave the **Source AMS Port** at `0`. The driver then adopts the number the local TCP port was assigned, which is different for every connection. A fixed value works as well, but only for a single device — see Several devices on the same PLC.

Device discovery needs the discovery agent. *Discover Devices* searches the networks the gateway itself is attached to, which for a container are Docker's, not the plant's. Run the ToddySoft discovery agent next to the container — see Running in Docker: the discovery agent — or enter the host and AMS Net ID by hand.

VPN related issues

When connecting to a remote PLC over a VPN, depending on the VPN type the host usually gets assigned a local VPN IP address. As with Docker, set a **Source AMS Net ID** that references this assigned IP address.

Leave the **Source AMS Port** at `0`. The driver then adopts the number the local TCP port was assigned, which is different for every connection. A fixed value works as well, but only for a single device — see Several devices on the same PLC.

Several devices on the same PLC

The TwinCAT router routes by **Source AMS Net ID**. When two devices announce the same one to the same PLC, the router treats the second connection as a replacement for the first and closes it. The two devices then keep evicting each other, which shows up as connections that repeatedly drop and return to `Connection Lost – see the Gateway logs for details`.

Give every device that connects to the same PLC its own **Source AMS Net ID**, and add a matching route on the PLC for each of them. Leaving **Source AMS Net ID** empty does not avoid the clash: the driver then derives it from the gateway's IP address, which is the same for every device on that gateway.

The module detects the clash and reports it in the Gateway log when the device starts:

```
[Device B] Configuration conflict: source AMS Net ID
'192.168.1.100.1.1' towards 192.168.1.10
is also used by Device A. The PLC only accepts one connection per
identity and will keep closing
whichever connection it considers the older one, so these devices
will repeatedly disconnect each
other. Give each device its own identity.
```

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.

- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.
- **New:** discovery when Ignition runs in Docker, through the ToddySoft discovery agent. See Running in Docker: the discovery agent.
- **Fixed:** a licence could be read as corrupted when Ignition delivered it while the gateway was running. The licence check ran on one of Ignition's own threads, which cannot see the module's files, so every licence, the bundled trial included, was rejected as *License file is corrupted*, and every connection after that was refused. The check now runs as the module.
- **Fixed:** the gateway log and the audit log no longer contain the licence itself. Only its name and length are logged.
- **Fixed: Discover Devices** lists each controller once. A broadcast leaves every network interface of the gateway, so a controller answered once per interface and showed up as several identical rows, and selecting one selected all of them. With a result selected, the **Apply** button also stays within the dialog.
- **Fixed:** one browsed tag the driver cannot type no longer stops the whole device. Until now the device never started and kept reconnecting with `Unknown plcValueType`, without naming the tag. The tag is now skipped and named in the gateway log (search for `Skipping browsed tag`), and the device starts.
- **Fixed:** browsing a large PLC program no longer fails on the request timeout. A browse makes many round trips, so it now has at least five minutes, never less than the request timeout.
- **Changed:** Ignition now refuses to load the module when its OPC UA module is missing or disabled, with a warning that names it, instead of loading it and failing with a `NoClassDefFoundError`. Ignition's own drivers behave the same way.
- **Changed:** the module now requires Ignition 8.3.0 instead of the development build `8.3.0-SNAPSHOT`, and its description on the gateway says it is built for Ignition 8.3, so it can be told apart from the 8.1 build once installed.
- **Changed:** date and time tags have new value types. DATE and DATE_AND_TIME are DateTime values (until now a DateTime tag holding epoch milliseconds as a number); TIME and LTIME are milliseconds, TIME_OF_DAY and LTIME_OF_DAY milliseconds since midnight (Int64). Writes take the same forms. Adjust scripts and bindings that used the old values.
- **Fixed:** a write the PLC refuses now fails in Ignition, with the reason as the status (for example `Bad_TypeMismatch` for a value of the wrong type). Until now every write was reported as successful, and the refusal appeared only in the gateway log.
- **Fixed:** a tag that has not been read yet, or whose last read failed, no longer reads 0 with Good quality. Before its first read it has no value and the quality Uncertain; after a failed read it keeps its last value with Bad quality until the next good read.
- **Fixed:** removing the last tag of a tag group (or polling rate) no longer leaves the device showing *Connection Lost*. A poll still running at that moment made the device fail with `Batch '...' not found` in the gateway log and stay in that state until it was restarted.

- **Fixed:** a tag addressed by index group and offset over a range of elements, for example `0x4040/600[0..5]:BYTE`, now reads every element; until now it read only the first. Symbolic addresses such as `MAIN.aValues` were not affected.
- **Changed:** a browsed array is one folder, named after the array and its index range (for example `arr[0..9]`), holding its elements; until now the elements sat directly beside the array's neighbours. A multi-dimensional array, until now left out of the browse, appears as nested folders down to its elements (`cube[1..2][1..2]`, then `cube[1][1..2]`, then `cube[1][1]`). An array of structures shows a folder per element holding its members (for example `pts[2].x`). Each element keeps its OPC item path, so existing tags keep working.

1.2.0 — 2026-09-11

Built on ToddySoft Connect 1.2.0.

- **Added: Discover Devices** — the device page can now broadcast on the gateway's networks and list the controllers that answer, grouped by device with a row per transport each one offers. Picking a result fills in host, port, AMS Net IDs and the transport; it fills only what the device actually reported, never a password, and asks first if that would overwrite something you had already typed. Requires permission to create device connections.
- **Added: Test Connection**, which asks the driver to diagnose the configuration as entered and says what is wrong rather than pass or fail. Where the driver can repair it, the diagnosis offers the fix: a Secure ADS device with no AMS route offers to register one, given the controller's administrator credentials — used for that one call and never stored. Where the fix cannot be automated, such as a TLS-PSK route, the steps to do it by hand are shown instead.
- **Added:** a **Machine Name** setting for the TLS transports. Secure ADS announces a machine name to the controller, and with a self-signed certificate it must equal the certificate's common name or the controller answers `CnMismatch`. Leave it empty to announce this gateway's host name without its domain.
- **Added:** the auto-generated keystore is now an explicit **Keystore** choice — *Auto-generated (module)* or *Custom path* — naming the file in use. A blank path always meant the module's own keystore; it simply looked unfilled. Nothing stored has changed.
- **Changed:** the **Ignore Common Name** setting has been removed. It never had any effect — the parameter behind it was read by nothing, because the ignore-CN flag exists only when *registering* a route, not when connecting. Nothing changes for a device that had it ticked. The same option now appears where it works, on the route registration offered by **Test Connection**, off by default.
- **Changed:** discovery no longer fills in **Source AMS Net ID** or **Source AMS Port**. Neither describes the device found — they describe this gateway's end, taken from whichever network interface happened to receive the broadcast reply, which on a gateway with several addresses is easily the wrong one and fails as a thirty-

second timeout. Left empty, the driver derives both from the connection it actually opens.

- **Fixed:** Secure ADS with the auto-generated certificate is no longer refused by every TwinCAT controller. The certificate was issued to this gateway's fully qualified host name while Secure ADS announces the short one, so the controller compared the two and answered `CnMismatch` after a successful handshake. The certificate is now issued to the short name, and a gateway holding a stale one regenerates it at startup. **A route registered against the old certificate must be registered again.**
- **Fixed:** a TLS device using the auto-generated keystore could not be created at all. The form demanded a **Keystore Path**, but an empty path is exactly how the module's own keystore is chosen — so **Create Device Connection** stayed greyed out, with no message saying which field was at fault, while **Test Connection** on the same page reported success.
- **Changed:** a browsed tag now shows the name of the device it came from — `Wall-1000_MAIN.temp` rather than `MAIN.temp`, folders included. Ignition names a tag after the node it was dragged from, so without this two devices both offered `MAIN.temp` and dragging from each into one folder collided. Only the displayed name changes: the OPC item path behind every tag is untouched, so **existing tags keep working** and only newly created ones take the new name. An array element, which used to show as just `[0]`, now reads `Wall-1000_MAIN.g_arr[0]`.
- **Fixed:** a device whose PLC was switched off no longer reports **Connected**. The startup reachability check ran and its result was thrown away; with tags configured the first failed poll corrected the status within seconds, so this was only visible on a device with no tags, which polls nothing and sat claiming *Connected* indefinitely. The status is now honest from the start and the device reconnects on its own once the PLC answers. A device with no tags is also checked every five minutes so an outage is noticed at all.
- **Fixed:** the individual nodes of a browsed array read the right element again. ToddySoft Connect 1.2.0 unified how arrays are written in an address, and an array's own bounds are now part of the address the driver reports; the module was still appending the element index to that, which asks for a second array dimension rather than for an element. An array declared with a lower bound other than zero — a TwinCAT `ARRAY [1..10]` — was also read one position out. Arrays with the usual zero lower bound were unaffected.
- **Fixed:** the **TLS-SCA (Shared Certificate Authority)** mode now presents this gateway's own certificate. It differs from TLS-SSC only in how the *PLC's* certificate is verified, so it was always meant to send the keystore — but only the self-signed mode ever did. A **Keystore Path** was collected, documented as required, and then never used, so a controller configured to demand a client certificate refused the connection. No configuration change is needed; a device already filled in starts working.

- **Fixed:** connecting over **TLS-PSK** no longer destroys the pre-shared key on the controller's route. Every Secure-ADS connect sent a route *registration* message carrying the PSK identity; TwinCAT took it as one, rewrote the route from a message whose format has no field for a key, and left the route's PSK empty — so an ordinary tag read wiped out the credential it had just authenticated with. The router keeps the route in memory, so nothing looked wrong until the controller was restarted, at which point the device could no longer connect and the route had to be entered again by hand. Fixed in ToddySoft Connect 1.2.0 and confirmed on hardware. If a route's PSK has already been emptied this way, re-enter it on the controller once; it will not be cleared again.
- **Changed (action required):** the **Route Name**, **Route Username** and **Route Password** settings have been removed. ToddySoft Connect 1.2.0 no longer registers an AMS route while connecting — that is now a separate, one-time administrative operation — so those three settings bound nothing and no route was ever registered from them. For a TLS/Secure-ADS device, create the route on the controller before it connects (TwinCAT XAE, or `/etc/TwinCAT/3.1/Target/StaticRoutes.xml`); a device pointed at a controller with no matching route will fail to connect. Since this release the module can register the route for you, from **Test Connection** on the device page — see *Registering an AMS route* above. Plain-TCP devices are unaffected.
- **Changed:** the keystore password and the pre-shared key are no longer written into the connection string. They are handed to the driver directly, so they no longer appear in the gateway log, in a support bundle, or in any string quoting the connection. Nothing changes on this page — the same fields, in the same places.
- **Added:** a **Limits** section with four ceilings on what the controller can make the driver allocate — browse depth, data-type table depth, and the two Sum-command sizes. The defaults are the driver's own, so an existing device connects exactly as before.

1.1.0

Built on ToddySoft Connect 1.1.0.

- **Fixed:** polling now recovers on its own after a long outage. A device whose PLC was switched off overnight, unreachable across a network outage, or idle while a licence was lapsed and later restored could keep failing every poll until the device was disabled and re-enabled by hand. The connection and tag polling are now rebuilt automatically once failures persist; existing OPC UA subscriptions are preserved.
- **Added:** three **Trust Store** settings, shown when the *TLS-SCA* transport is selected, so a PLC certificate signed by your own CA can be validated. Previously only certificate authorities already trusted by the Java runtime could be used. Leaving **Trust Store Path** empty keeps the previous behaviour.
- **Fixed:** ADS connection checks now use the configured AMS port, so a runtime on a non-default AMS port is no longer reported as unreachable while data still flows.

1.0.0

Initial release.